CSA1709 - Artificial Intelligence - SLOT: B

LAB EXPERIMENTS

Experiment No: 17

Write a Prolog Program to Sum the Integers from 1 to n

Aim

To write and execute a Prolog program to calculate the sum of all integers from 1 to n using recursion.

Algorithm

1. Start the program.
2. Define the base case: if N = 0, then the sum is 0.
3. Define the recursive case:
    - Check whether N > 0.
    - Decrease the value of N by 1.
    - Recursively calculate the sum up to N − 1.
    - Add N to the previous sum.
4. Return the final sum.
5. Stop.

Program (GNU Prolog)

```
% Program to find the sum of integers from 1 to N

sum(0, 0).

sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
```

S is S1 + N.

Input (Query)

?- sum(5, S).

Output

S = 15.

yes

Another Example

Input

?- sum(10, S).

Output

S = 55.

yes

```
| ?- consult('C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl').
compiling C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl compiled, 8 lines read - 874 bytes written, 9 ms

(15 ms) yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl').
compiling C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/sum of integers from 1 to N.pl compiled, 8 lines read - 874 bytes written, 11 ms

yes
| ?- sum(10,S).

S = 55 ?
```

Result

Thus, the Prolog program to calculate the sum of integers from 1 to n was successfully implemented and executed using recursion. The program correctly computes the sum for the given value of n.

Experiment No: 18

Write a Prolog Program for a Database with Name and Date of Birth (DOB)

Aim

To create a simple database in Prolog that stores a person's **Name** and **Date of Birth (DOB)** and retrieve the information using queries.

Algorithm

1. Start the program.
2. Define a predicate person(Name, DOB) to store the database.
3. Enter the details of each person as facts.
4. Save the program with the extension .pl.
5. Load the program in GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the results.
8. Stop the program.

Program (GNU Prolog)

```
% Name and DOB Database

person(kaviya, '15-08-2005').
person(arun, '10-02-2004').
person(priya, '25-12-2003').
person(rahul, '05-06-2002').
person(meena, '30-09-2004').
```

Sample Input 1

?- person(kaviya, DOB).

Output

DOB = '15-08-2005'.

yes

```
| ?- Database of Name and Date of Birth.pl').
uncaught exception: error(syntax_error('user_input:4 (char:299) . or operator expected after expression'),read_term/3)
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl compiled, 6 lines read - 795 bytes written, 9 ms

(16 ms) yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Database of Name and Date of Birth.pl compiled, 6 lines read - 795 bytes written, 9 ms

(31 ms) yes
| ?- person(kaviya, DOB).

DOB = '15-08-2005'

yes
| ?- person(priya,DOB)

uncaught exception: error(syntax_error('user_input:6 (char:1) . or operator expected after expression'),read_term/3)
| ?- person(priya,DOB).

DOB = '25-12-2003'

yes
```

Result

Thus, the Prolog program for creating a **Name and Date of Birth (DOB) database** was successfully implemented and executed. The required information was retrieved correctly using Prolog queries.

Experiment No: 19

Title

Write a Prolog Program for Student–Teacher–Subject–Code Database

Aim

To write and execute a Prolog program to create a database containing **Student Name, Teacher Name, Subject, and Subject Code**, and retrieve information using queries.

Algorithm

1. Start the program.
2. Define a predicate student_teacher_subject_code(Student, Teacher, Subject, Code).
3. Store the student, teacher, subject, and subject code details as facts.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the result.
8. Stop the program.

Program (GNU Prolog)

```
% Student-Teacher-Subject-Code Database


student_teacher_subject_code(kaviya, ram, artificial_intelligence, cs301).
student_teacher_subject_code(arun, priya, dbms, cs302).
student_teacher_subject_code(meena, kumar, operating_system, cs303).
student_teacher_subject_code(rahul, lakshmi, computer_networks, cs304).
student_teacher_subject_code(divya, ravi, java_programming, cs305).
```

Input (Query 1)

```
?- student_teacher_subject_code(kaviya, Teacher, Subject, Code).
```

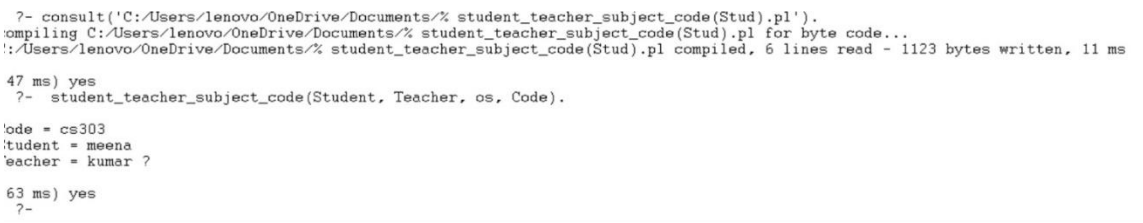
Output

Teacher = ram,

Subject = artificial_intelligence,

Code = cs301.

Yes



Result

Thus, the Prolog program for creating a **Student–Teacher–Subject–Code Database** was successfully implemented and executed. The required information was retrieved correctly using Prolog queries.

Experiment No: 20

Title

Write a Prolog Program for Planets Database

Aim

To write and execute a Prolog program to create a **Planets Database** containing the **planet name, type, and position from the Sun**, and retrieve information using queries.

Algorithm

1. Start the program.
2. Define a predicate planet(Name, Type, Position).

3. Store the details of the planets as facts.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries to retrieve the required information.
7. Display the result.
8. Stop the program.

Program (GNU Prolog)

% Planets Database

planet(mercury, terrestrial, 1).

planet(venus, terrestrial, 2).

planet(earth, terrestrial, 3).

planet(mars, terrestrial, 4).

planet(jupiter, gas_giant, 5).

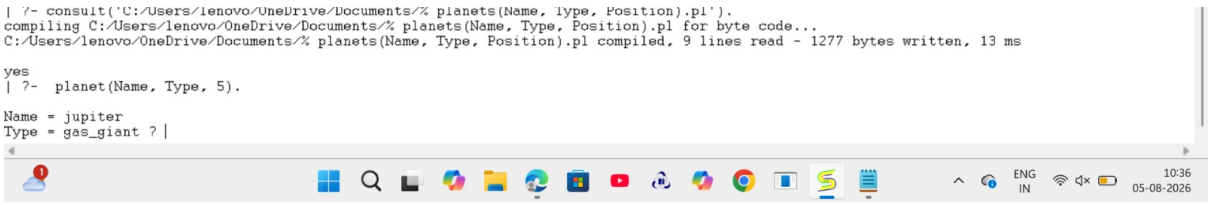
planet(saturn, gas_giant, 6).

planet(uranus, ice_giant, 7).

planet(neptune, ice_giant, 8).

Input (Query 1)

?- planet(earth, Type, Position).



Result

Thus, the Prolog program for creating a **Planets Database** was successfully implemented and executed. The details of the planets were stored and retrieved correctly using Prolog queries.

Experiment No: 21

Title

Write a Prolog Program to Implement Towers of Hanoi

Aim

To write and execute a Prolog program to solve the **Tower of Hanoi** problem using recursion and display the sequence of disk movements.

Algorithm

1. Start the program.
2. Define the base case:
    - If there is only one disk, move it directly from the source peg to the destination peg.
3. Define the recursive case:
    - Move **N−1** disks from the source peg to the auxiliary peg.
    - Move the **Nth** disk from the source peg to the destination peg.
    - Move the **N−1** disks from the auxiliary peg to the destination peg.
4. Repeat the process until all disks are moved.
5. Display each move.
6. Stop the program.

Program (GNU Prolog)

```
% Tower of Hanoi Program


hanoi(1, Source, Destination, _) :-
    write('Move disk 1 from '),
    write(Source),
    write(' to '),
    write(Destination), nl.
```

```
hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    N1 is N - 1,
    hanoi(N1, Source, Auxiliary, Destination),
    write('Move disk '),
    write(N),
    write(' from '),
    write(Source),
    write(' to '),
    write(Destination), nl,
    hanoi(N1, Auxiliary, Destination, Source).
```

```
    hanoi(N1, Auxiliary, Destination, Source).
uncaught exception: error(existence_error(procedure,(:-)/2),top_level/0)
| ?- ?- hanoi(3, left, right, center).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?- hanoi(3, left, right, center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right

true ? ?
Action (; for next solution, a for all solutions, RET to stop) ? -
Action (; for next solution, a for all solutions, RET to stop) ? ;

no
| ?- hanoi(2, a, c, b).
Move disk 1 from a to b
Move disk 2 from a to c
Move disk 1 from b to c

true ?
```

Result

Thus, the Prolog program to implement the **Tower of Hanoi** was successfully executed. The program correctly displayed the sequence of moves required to transfer all disks from the source peg to the destination peg using the auxiliary peg.

Experiment No: 22

Title

Write a Prolog Program to Print Whether a Particular Bird Can Fly or Not. Incorporate Required Queries.

Aim

To write and execute a Prolog program to determine whether a particular bird can fly or not using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Create rules to identify whether a bird can fly or not.
5. Save the program with the extension .pl.
6. Load the program into GNU Prolog.
7. Execute queries for different birds.
8. Display whether the bird can fly or cannot fly.
9. Stop the program.

Program (GNU Prolog)

```
% Birds that can fly

can_fly(parrot).

can_fly(pigeon).

can_fly(sparrow).

can_fly(crow).

can_fly(peacock).
```

% Birds that cannot fly

cannot_fly(ostrich).

cannot_fly(penguin).

cannot_fly(emu).

cannot_fly(kiwi).

Input (Query 1)

?- can_fly(parrot).

Output

yes

```
| ?- Birds that can fly.Pl').
uncaught exception: error(syntax_error('user_input:30 (char:185) . or operator expected after expression'),read_term/3)
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl compiled, 11 lines read - 896 bytes written, 4 ms

yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Birds that can fly.Pl compiled, 11 lines read - 896 bytes written, 10 ms

yes
| ?-  cannot_fly(ostrich).

yes
| ?- ?- cannot_fly(nova).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?-  can_fly(penguin).

no
| ?-
```

Result

Thus, the Prolog program to determine whether a particular bird can fly or not was successfully implemented and executed. The program correctly identified birds that can fly and birds that cannot fly using Prolog facts and queries.

Experiment No: 23

Title

Write a Prolog Program to Implement Family Tree

Aim

To write and execute a Prolog program to represent a **family tree** using facts and rules, and retrieve family relationships through queries.

Algorithm

1. Start the program.
2. Define the facts for **male**, **female**, and **parent** relationships.
3. Create rules to determine **father**, **mother**, **brother**, **sister**, **grandparent**, and **grandchild** relationships.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries to find different family relationships.
7. Display the results.
8. Stop the program.

Program (GNU Prolog)

```
% Family Tree Program

% Male members
male(john).
male(robert).
male(david).
male(james).

% Female members
female(mary).
```

```
female(linda).
female(susan).
female(emily).

% Parent relationships
parent(john, robert).
parent(mary, robert).
parent(john, linda).
parent(mary, linda).
parent(robert, david).
parent(susan, david).
parent(robert, emily).
parent(susan, emily).

% Rules
father(X, Y) :-
    male(X),
    parent(X, Y).

mother(X, Y) :-
    female(X),
    parent(X, Y).

brother(X, Y) :-
    male(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

```
sister(X, Y) :-
    female(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.


grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).


grandchild(X, Y) :-
    grandparent(Y, X).
```

Input (Query 1)

```
?- father(X, david).
```

Output

```
| ?- mily Tree Program.pl').
uncaught exception: error(syntax_error('user_input:31 (char:270) . or operator expected after expression'),read_term/3)
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Family Tree Program.pl compiled, 50 lines read - 3699 bytes written, 11 ms

yes
| ?- ?- father(X, david).
uncaught exception: error(existence_error(procedure,(?-)/1),top_level/0)
| ?-  father(X, david).

X = robert ? y
```

Result

Thus, the Prolog program to implement a Family Tree was successfully executed. The program correctly represented family relationships and retrieved information such as father, mother, brother, sister, grandparent, and grandchild using Prolog facts, rules, and queries

Experiment No: 24

Title

Write a Prolog Program to Suggest a Dieting System Based on Disease

Aim

To write and execute a Prolog program that suggests a suitable diet based on a patient's disease using facts and rules.

Algorithm

1. Start the program.
2. Define the diseases and their corresponding diet recommendations as facts.
3. Store each disease with its recommended diet.
4. Save the program with the extension .pl.
5. Load the program into GNU Prolog.
6. Execute queries by entering the disease name.
7. Display the recommended diet.
8. Stop the program.

Program (GNU Prolog)

```
% Diet Recommendation System

diet(diabetes, 'Low sugar diet').
diet(hypertension, 'Low salt diet').
diet(obesity, 'Low fat diet').
diet(anemia, 'Iron rich diet').
diet(fever, 'Liquid diet').
diet(ulcer, 'Soft and bland diet').
diet(kidney_disease, 'Low protein diet').
diet(heart_disease, 'Low cholesterol diet').
```

Input (Query 1)

?- diet(diabetes, Diet).

Output

```
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl compiled, 9 lines read - 1217 bytes written, 7 ms

(15 ms) yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Diet Recommendation System.pl compiled, 9 lines read - 1217 bytes written, 10 ms

yes
| ?-  diet(hypertension, Diet).

Diet = 'Low salt diet'

yes
| ?- diet(fever, Diet).

Diet = 'Liquid diet'

yes
```

Result

Thus, the Prolog program to **suggest a dieting system based on disease** was successfully implemented and executed. The program correctly recommended the appropriate diet for the given disease using Prolog facts and queries.

Experiment No: 25

Title

Write a Prolog Program to Implement the Monkey Banana Problem

Aim

To write and execute a Prolog program to solve the **Monkey Banana Problem** using facts and rules, where the monkey performs actions to obtain the banana.

Algorithm

1. Start the program.
2. Define the initial state:
    - Monkey is on the floor.
    - Monkey is at the door.
    - Box is at the window.
    - Banana is hanging from the center.
3. Define the actions:
    - Move the monkey to the box.
    - Push the box to the banana.
    - Climb onto the box.
    - Grasp the banana.
4. Execute the actions in sequence.
5. Display each action performed by the monkey.
6. Stop the program.

Program (GNU Prolog)

```
% Monkey Banana Problem
move :-
    write('Step 1: Monkey goes from the door to the box.'), nl,
    write('Step 2: Monkey pushes the box to the banana.'), nl,
    write('Step 3: Monkey climbs onto the box.'), nl,
```

```
    write('Step 4: Monkey grasps the banana.'), nl,
    write('Monkey successfully gets the banana!'), nl.
```

Input (Query)

?- move.

Output

```
yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl compiled, 7 lines read - 1086 bytes written, 9 ms

yes
| ?- consult('C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl').
compiling C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl for byte code...
C:/Users/lenovo/OneDrive/Documents/% Monkey Banana Problem.pl compiled, 7 lines read - 1086 bytes written, 10 ms

yes
| ?- move.
Step 1: Monkey goes from the door to the box.
Step 2: Monkey pushes the box to the banana.
Step 3: Monkey climbs onto the box.
Step 4: Monkey grasps the banana.
Monkey successfully gets the banana!

yes
| ?-
```

Result

Thus, the Prolog program to implement the **Monkey Banana Problem** was successfully executed. The program correctly displayed the sequence of actions required for the monkey to obtain the banana.